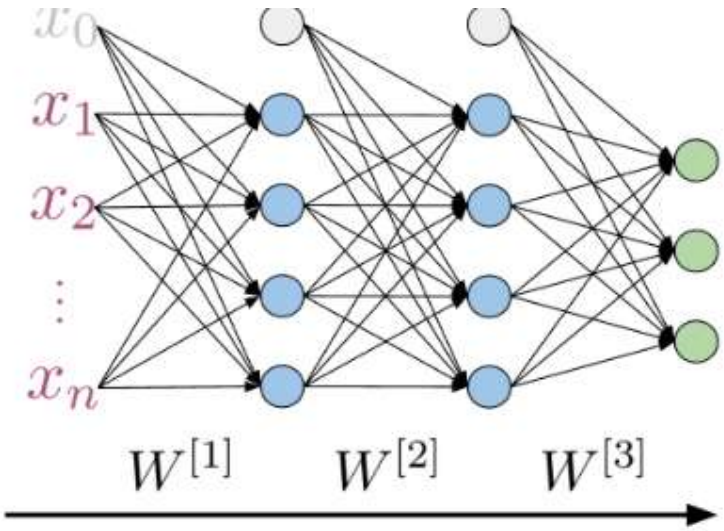
NLP- Sequential Models

Neural Networks for Sentiment Analysis

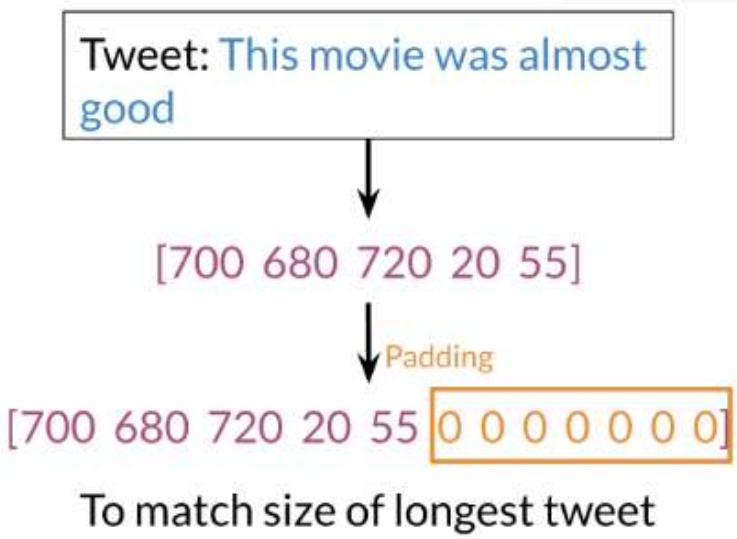


$$a^{[0]} = X$$

$$z^{[i]} = W^{[i]} a^{[i-1]}$$

$$a^{[i]} = g^{[i]}(z^{[i]})$$

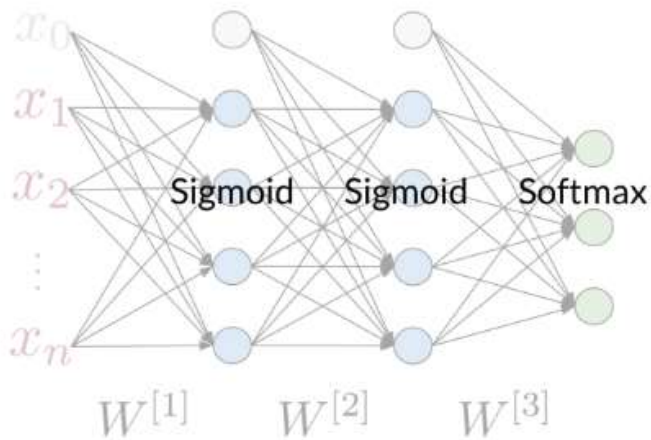
Word	Number
a	1
able	2
about	3
...	...
hand	615
...	...
happy	621
...	...
zebra	1000



One of the Deep-Learning (NLP) Framework built on top of Tensorflow is **Trax**. It has several advantages:

1. It supports parallel computing
2. Runs fast on CPU,GPU and TPU

3. Records algebriac computation for gradient evaluation.




```
from trax import layers as tl
Model = tl.Serial(
        tl.Dense(4),
        tl.Sigmoid(),
        tl.Dense(4),
        tl.Sigmoid(),
        tl.Dense(3),
        tl.Softmax())
```

For more :

- https://trax-ml.readthedocs.io/en/latest/
- https://github.com/google/trax

Embedding and Mean Layers

An Embedding layer creates index for each word in the Vocabulary and creates determined dimensions(here 2) of vectors.Every value that the embedding layer has created are Trainable in Trax.
The size of the weight matrix = Size of Vocabulary*Size of Embedding Layers

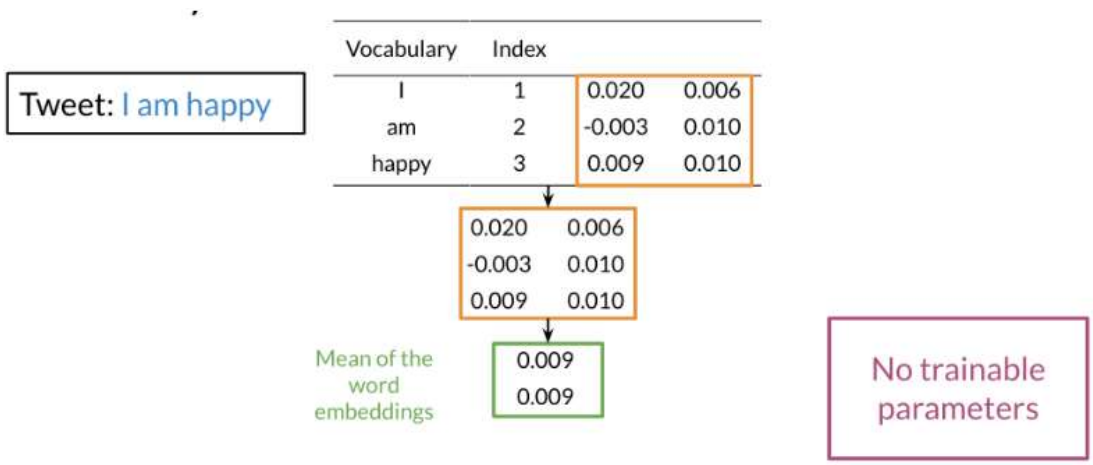
Vocabulary	Index		
I	1	0.020	0.006
am	2	-0.003	0.010
happy	3	0.009	0.010
because	4	-0.011	-0.018
learning	5	-0.040	-0.047
NLP	6	-0.009	0.050
sad	7	-0.044	0.001
not	8	0.011	-0.022

Trainable weights

Vocabulary x Embedding

For the tweet "I am happy", we of embedding size 2 we get the size of the trainable weights = 3*2 but if there are padding layers present so the size of the trainable weights will increase to a large extent and we will end up with a lot of parameters to train.

To counter this problem we take the mean of each embedding vectors and carry that forward to the sequential layers. The mean layer do not contain any trainable parameter.



Gradient in Trax

$$f(x) = 3x^2 + x$$

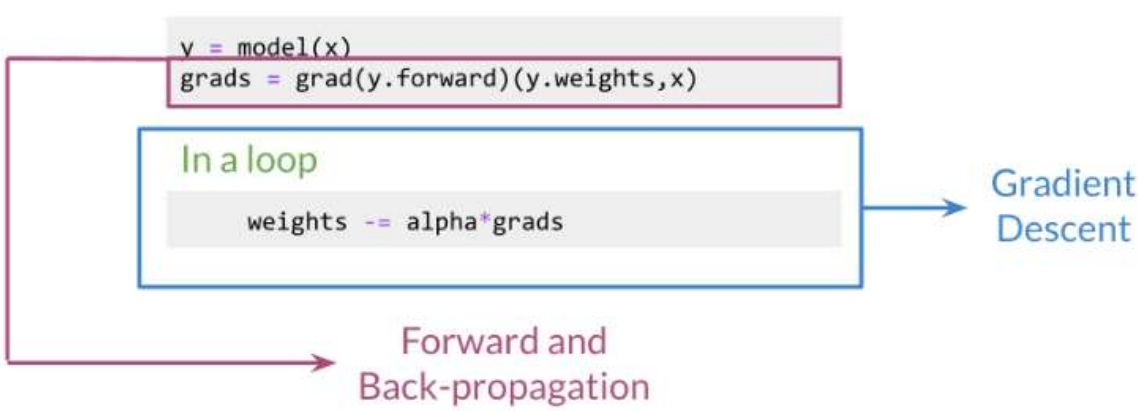
$$\boxed{\frac{\delta f(x)}{\delta x}} = 6x + 1$$

Gradient

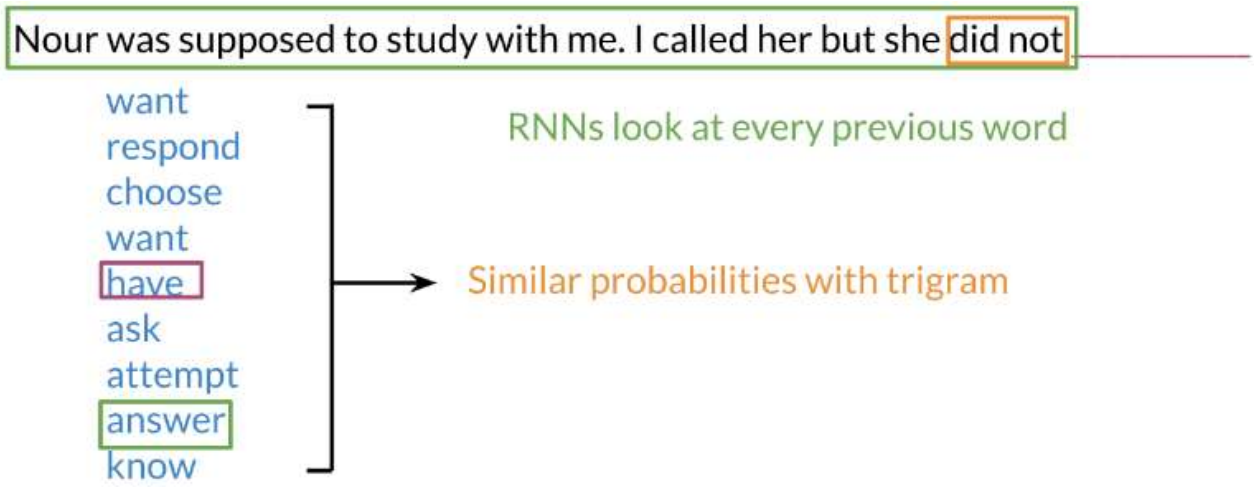
```
def f(x):
    return 3*x**2 + x

grad_f = trax.math.grad(f)
```

Returns a function

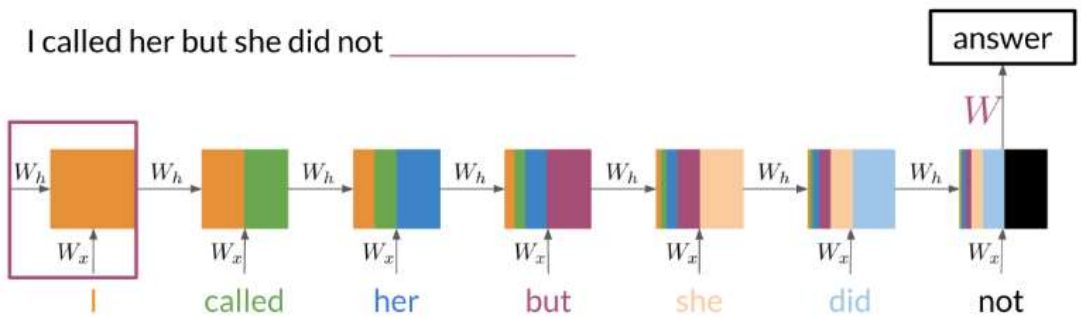
For the Neural Networks:



Recurrent Neural Network



An N-gram (trigram) would only look at "did not" and would try to complete the sentence from there. As a result, the model will not be able to see the beginning of the sentence "I called her but she". Probably the most likely word is have after "did not". RNNs help us solve this problem by being able to track dependencies that are much further apart from each other. As the RNN makes its way through a text corpus, it picks up some information as follows



Note that as you feed in more information into the model, the previous word's retention gets weaker, but it is still there. Look at the orange rectangle above and see how it becomes smaller as you make your way through the text. This shows that your model is capable of capturing dependencies and remembers a previous word although it is at the beginning of a sentence or paragraph. Another advantage of RNNs is that a lot of the computation shares parameters

- RNN propagates informaion from the start of a sentence through to the end.
- Then it propagates some of the computed information , takes the 2nd element in the sequence and generate new values(computed information).
- Then again it propagates some of the computed information from the first and second steps, and takes the 3rd element in the sequence and computes new information....and the process goes on till the last element in the sentence.
- In RNN the information from each word in the sequence is multiplied by the same weight W_x and the information propagated from first element of sequence through to the last element of the sequence is multiplied by W_h

- So in RNN the learning parameters are W_x,W_h and W, weights used to predict the final word.
- An RNN would have the same number of parameters for word sequences of different lengths

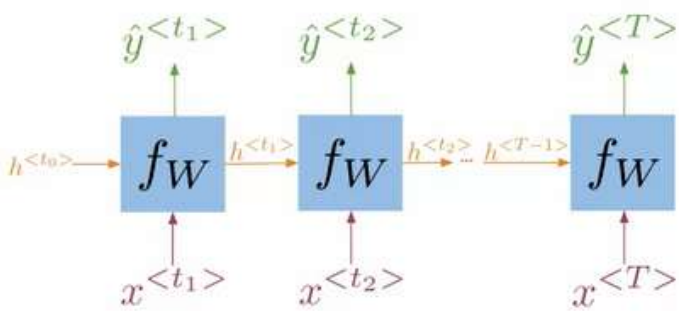
Applications of RNN

- One to One: given some scores of a championship, you can predict the winner
- One to many : Picture to caption generation. To describe a picture multiple words are used
- Many to one : Sentiment analysis. To describe the sentiment of a person a tweet consisting multiple words are used.
- Many to Many : Machine Translation. English to Bengali translation is one of the example. Multiple words taken as input , in output also we get multiple words.Encoders and decoders are used in this usecase.

Mathematical Interpretation behind RNN

Here our point of focus will be to understand how RNN propagate information from the first word to the last word of the sequence through Time and How they make predictions.Hidden states propagates information through Time.
Each Neuron of an RNN has 2 inputs -

1. Output of the previous hidden state $h^{<t-1>}$
2. Value of input at time t, can be a word at time t denoted by $x^{<t>}$



$$h^{<t>} = g(W_h[h^{<t-1>}, x^{<t>}] + b_h)$$
$$\hat{y}^{<t>} = g(W_{yh}h^{<t>} + b_y)$$

$$\boxed{h^{<t>} = g(W_{hh}h^{<t-1>} + W_{hx}x^{<t>} + b_h)}$$

Note that:

$$h^{<t>} = g(W_h[h^{<t-1>}, x^{<t>}] + b_h)$$

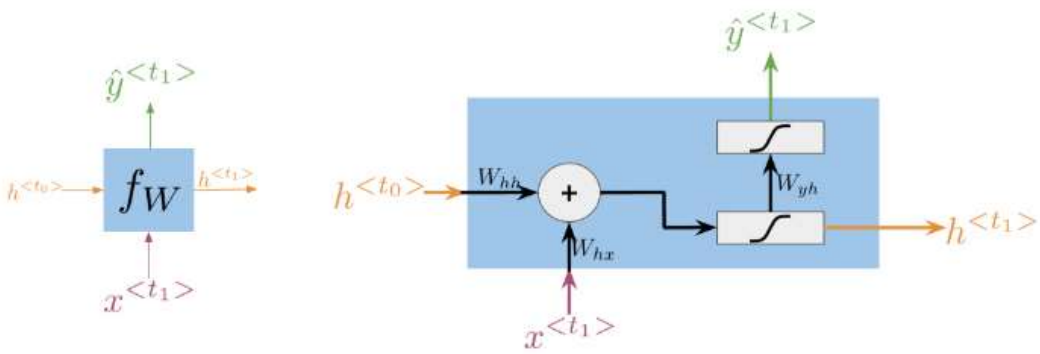
Is the same as multiplying W_{hh} by h and W_{hx} by x. In other words, you can concatenate it as follows:

$$h^{<t>} = g\left(W_{hh}h^{<t-1>} \oplus W_{hx}x^{<t>} + b_h\right)$$

For the prediction at each time step, you can use the following:

$$\hat{y}^{<t>} = g\left(W_{yh}h^{<t>} + b_y\right)$$

Note that you end up training $W_{hh}, W_{hx}, W_{yh}, b_h, b_y$. Here is a visualization of the model.



Cost function for RNN

Cross Entropy Loss=Log Loss = Loss of one single training Example=

$J^t = -\sum_{k=1}^{N} y_k log\hat{y}_k$

Where N = Number of output classes

Mean of the Loss for each batch of input to the model in each epoch =Time Average of the losses=

$J_T = \frac{1}{T}\sum_{t=1}^{T} J^t = -\frac{1}{T}\sum_{t=1}^{T}\sum_{k=1}^{N} y_k^{<t>} log\hat{y}_k^{<t>}$

y= actual value of centre word

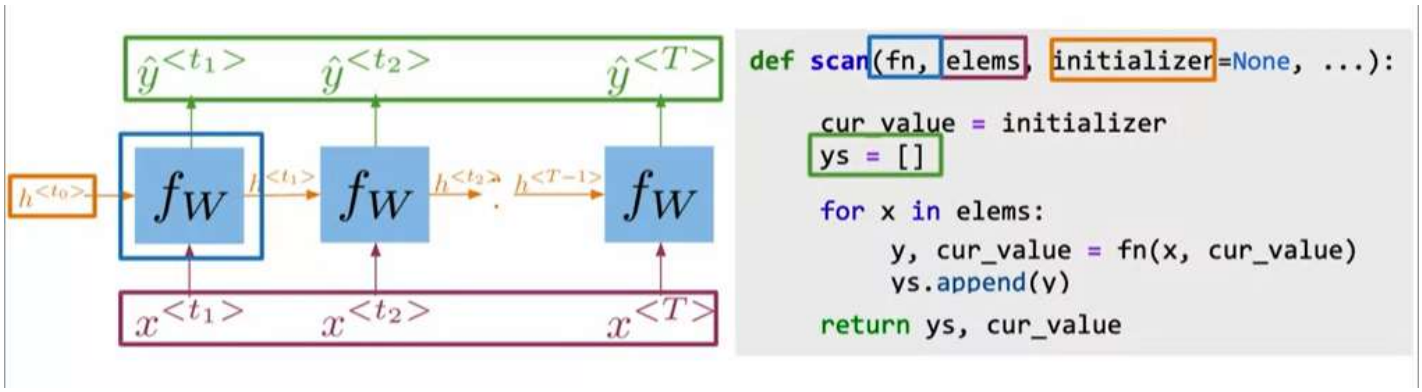
$\hat{y}$= Predicted value of y

NOTE : we are simply summing over all the time steps and dividing by T, to get the average cost in each time step. Hence, we are just taking an average through time

Implementation Note

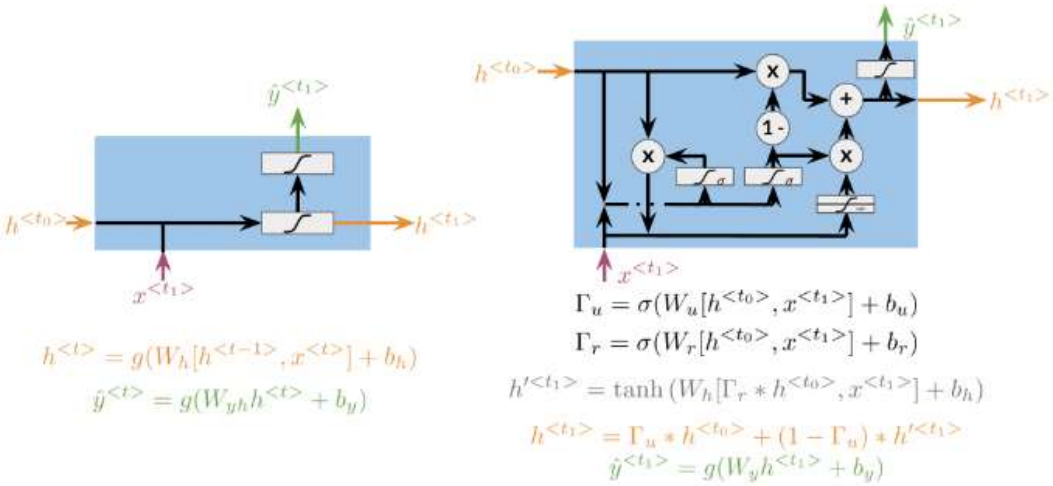
Scan function in Tensorflow is used to carry forward forward propagation using abstraction.Tensorflow need this type of function to perform parallel computation and run on GPU. It takes the initializer, and returns a list of outputs (ys), and uses the current value, to get the next y and the next current value

tf.scan() function



Gated Recurrent Unit (GRU)

Gated recurrent units are very similar to vanilla RNNs, except that they have a "relevance" and "update" gate that allow the model to update and get relevant information



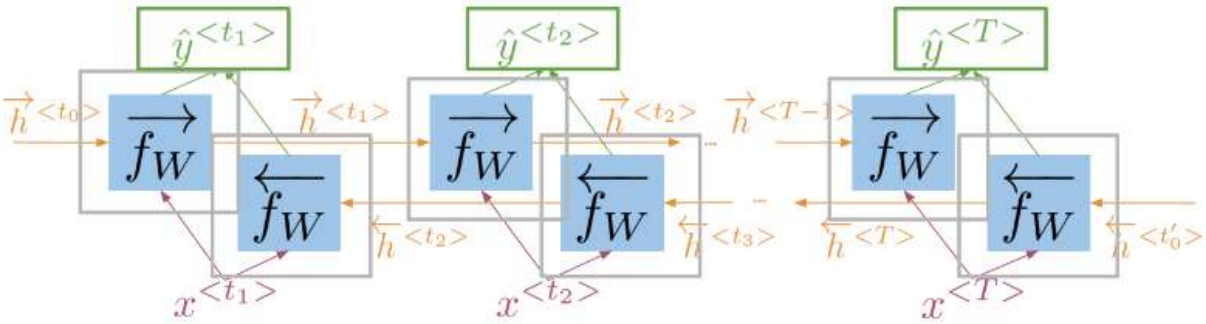
To the left, you have the diagram and equations for a simple RNN. To the right, we explain the GRU. Note that we add 3 layers before computing h and y.

$$\Gamma_u = \sigma\left(W_u\left[h^{<t_0>}, x^{<t_1>}\right] + b_u\right)$$
$$\Gamma_r = \sigma\left(W_r\left[h^{<t_0>}, x^{<t_1>}\right] + b_r\right)$$
$$h'^{<t_1>} = \tanh\left(W_h\left[\Gamma_r * h^{<t_0>}, x^{<t_1>}\right] + b_h\right)$$

The first gate Γ_u allows you to decide how much you want to update the weights by. The second gate Γ_r, helps you find a relevance score. You can compute the new hh by using the relevance gate. Finally you can compute h, using the update gate. GRUs “decide” how to update the hidden state. GRUs help preserve important information.

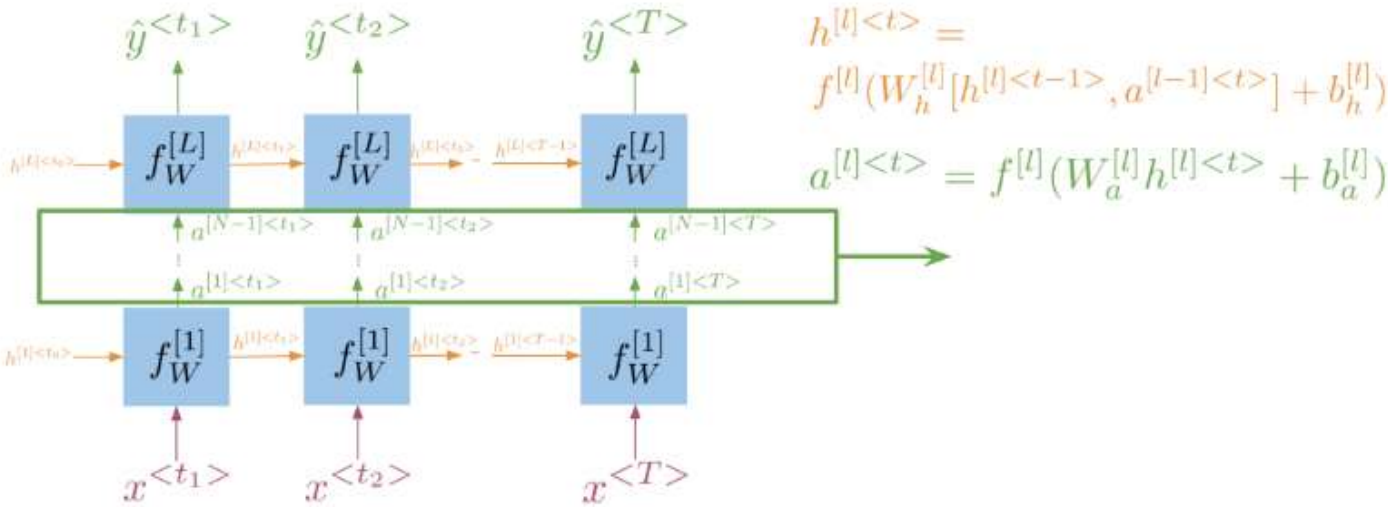
Bi-directional RNN

Bi-directional RNNs are important, because knowing what is next in the sentence could give you more context about the sentence itself.
here, Information flows from the past and from the future independently.
Here, $\hat{y}^{<t>} = g(W_y[\overrightarrow{h^{<t>}}, \overleftarrow{h^{<t>}}] + b_y)$



Deep RNN

Deep RNN are nothing but simple RNNs stacked together.
Intermediate connections pass information through the values of activation in each timestamp just like a conventional DNN.



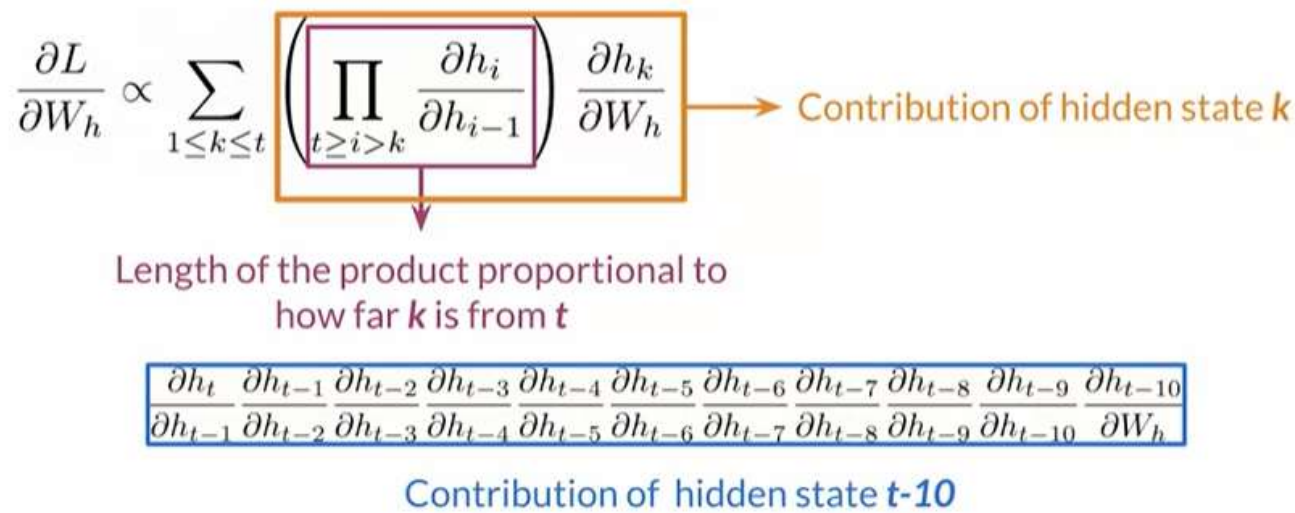
at layer l, you are using the input from the bottom a^{l-1} and the hidden state h^l That allows you to get your new h, and then to get your new a, you will train another weight matrix Wa , which you will multiply by the corresponding h add the bias and then run it through an activation layer

Vanishing & Exploding gradients in RNN

> Advantages of RNN:
>
> - Consumes less RAM as compared to other N-Gram models
> - Captures dependencies with short range.

> Disadvantages:
>
> - Struggles to capture long term dependencies
> - Prone to Vanishing and Exploding gradient problems.

Back-propagation through Time

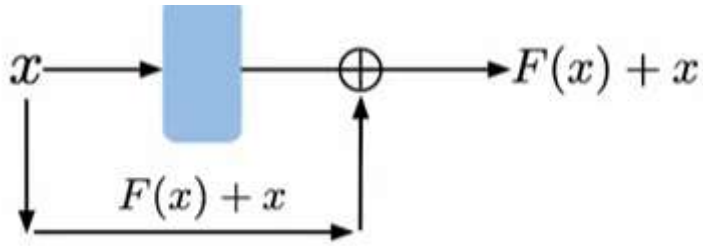


If the Partial Derrivative has value <1 then due to the multiplication of the terms the gradient value becomes very small and we face the vanishing gradient problem.
Similarly, if the partial derrivative has values > 1 then we face the Exploding gradient problem.

Countermeasures to tackle the problem

1. Identity RNN and ReLU activation : It's useful only for the vanishing gradient problems where the weights are initialized to the identity matrix and along with that ReLU activation function is used.
2. Gradient Clipping : This method is useful for the Expploding gradient issue where we can clip the values of the gradient passing a certain threshold to that given threshold. Say we can make all the gradient values greater than 100 to 100.
3. Skip-Connection : It enabled the RNN model to skip connection from its activation function to create a direct connect with the previous input (x). In this way activations from earlier layers

have more influence at the cost.



Reference Link : https://blog.paperspace.com/intro-to-optimization-in-deep-learning-gradient-descent/

Long Short-Term Memory (LSTM)

LSTM is a special variety of RNN which is able to remember only the relevant information and forget the irrelevant ones.
Theere are multiple gates in LSTM unlike in GRU there are only 2.
LSTM has a **cell-state** which works as a the memory of the RNN and a **hidden state** where the computations are performed.
A typical LSTM has 3 gates : Forget Gate , Input Gate , Output Gate(In this order).The gates allow the gradients to flow unchanged.

- Forget gate: tells you how much information to forget at any time point. It makes use of the previous hidden state $h^{<t_0>}$ & the input $x^{<t_1>}$.It decides which informations to keep.
- Input gate: tells you how much information to input at any time point. It decides which informations to add.
- Output gate: tells you how much information to pass over at any time point.It decides what the next hidden state will be.

Equations

The forget gate: $f = \sigma\left(W_f\left[h_t{-}_1; x_t\right] + b_f\right)$

The input gate: $i = \sigma\left(W_i\left[h_t{-}_1; x_t\right] + b_i\right)$

The gate gate: $g = \tanh\left(W_g\left[h_t{-}_1; x_t\right] + b_g\right)$

The cell state: $c_t = f \odot c_{t-1} + i \odot g$

The output gate: $o = \sigma\left(W_o\left[h_{t-1}; x_t\right] + b_o\right)$

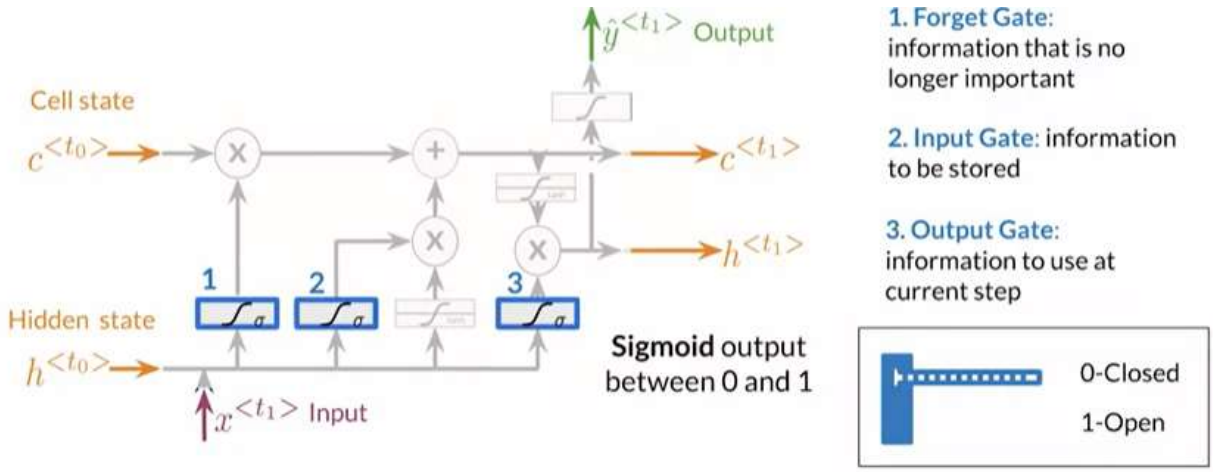
The output of LSTM unit: $h_t = o_t + tanh(c_t)$

Applications of LSTM:

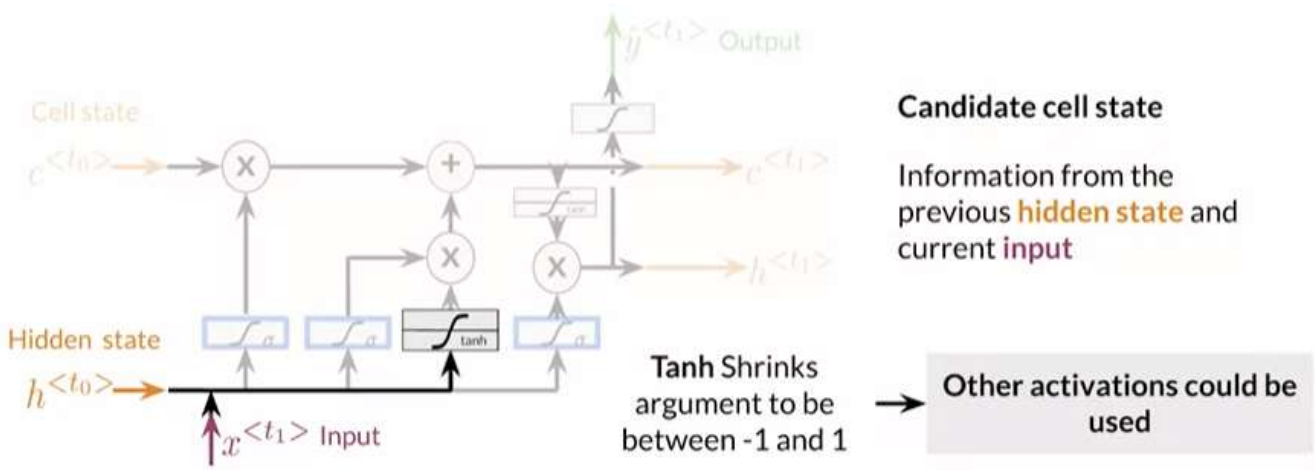
- Speech Recognition
- Next Character Prediction
- Music Composition
- Chatbot
- Image captioning

LSTM Architecture

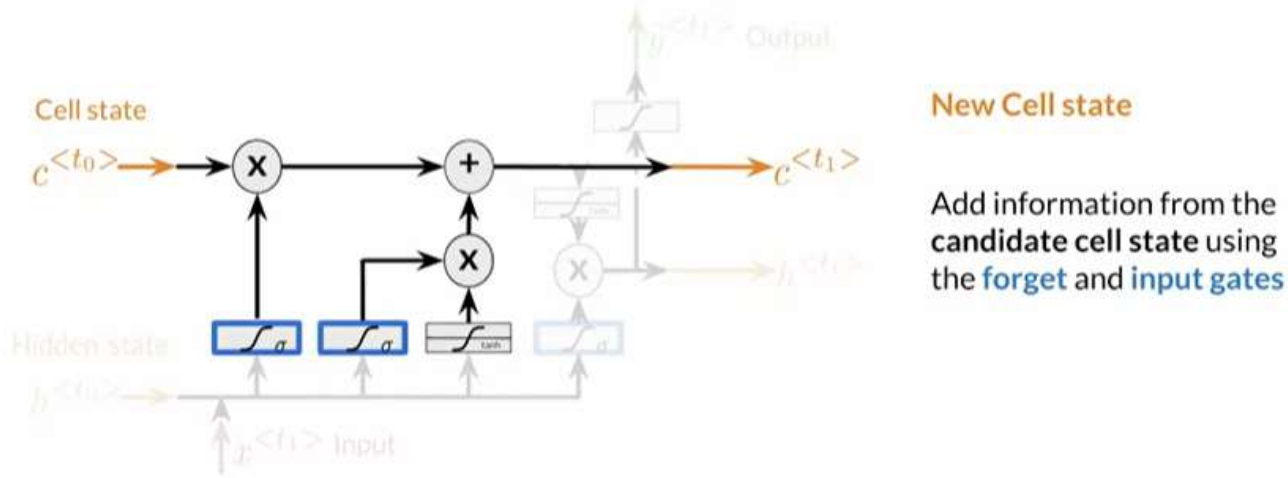
Gates in LSTM



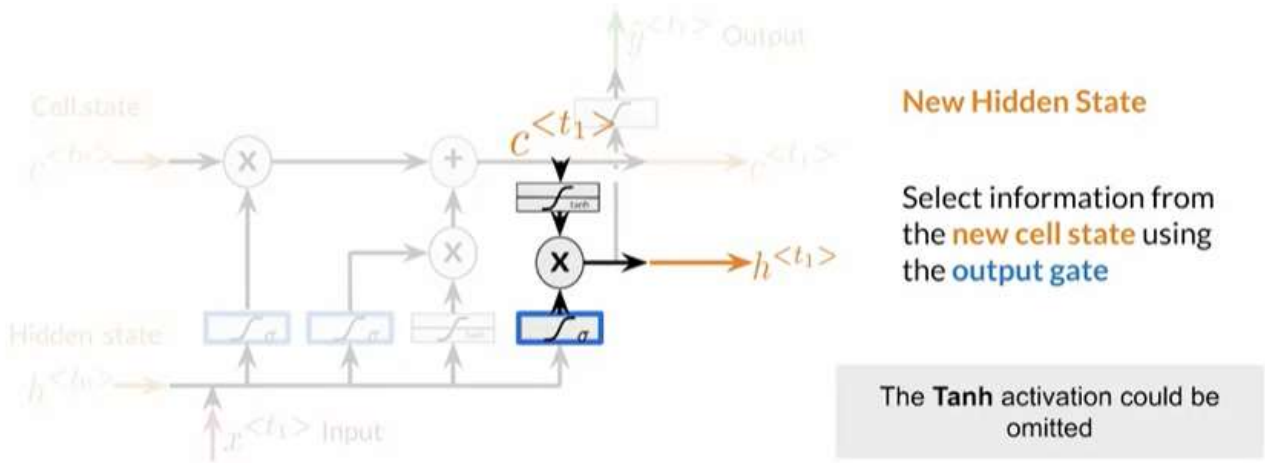
Candidate Cell-State



New-Cell State



New Hidden state



Named Entity Recognition use-case

NER systems are being used in search efficiency, recommendation engines, customer service, automatic trading, and many more.

Data Preprocessing

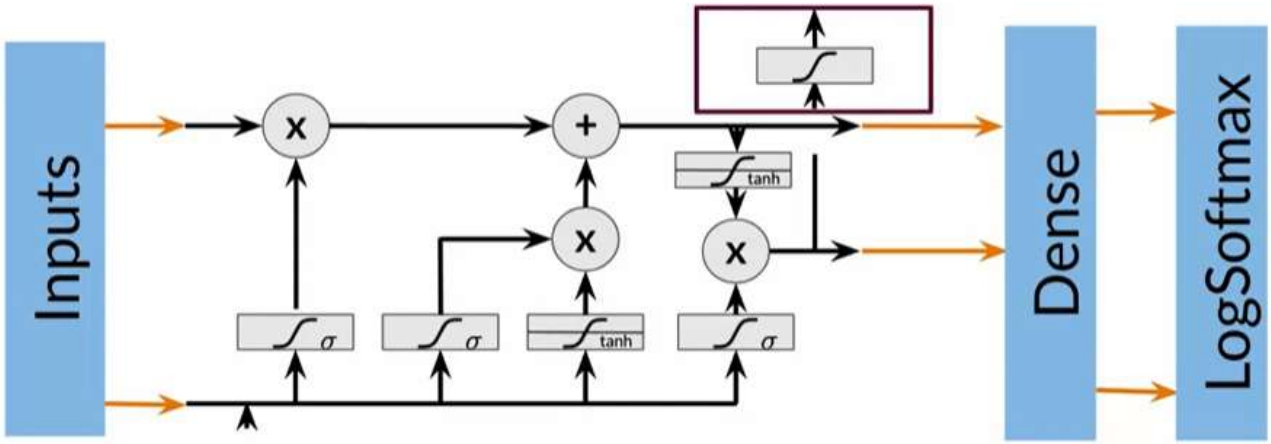
> 1. Assign each class a number.
> For example A geographical indicator can be labeled as 1, a person can be labeled as 2, a time indicator can be labeled as 3 .....etc
> 2. Assign each word a number.
> The whole sequence has to be labeled with a corresponding integer value.
> 3. Padding
> In LSTM all the sequences has to be of same size. To do that we can set the

> sequence length to a certain number and use <PAD> token to fill the empty space.

Training the NER

The training process goes though the following methods-

1. Create a Tensor for each input and it's corresponding number.
2. Put them in a Batch
3. Feed it into LSTM unit
4. Run the output through a Dense Layer.
5. Predict using log Softmax over K classes.




```
model = tl.Serial(
    tl.Embedding(),
    tl.LSTM(),
    tl.Dense()
    tl.LogSoftmax()
  )
```

Model Evaluation

For evaluation of the LSTM model the following steps are to be performed:

- Pass the test set through the model.
- get argmax across the prediction array
- Mask the padded tokens

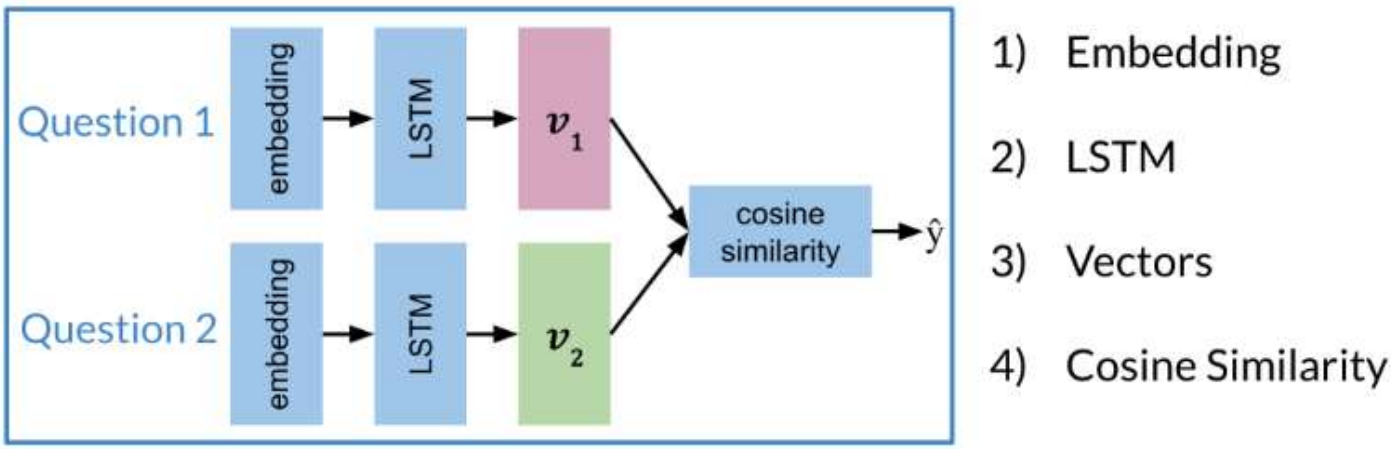
- Compare outputs against test labels.

```
def evaluate_model(test_sentences, test_labels, model):
    pred = model(test_sentences)
    outputs = np.argmax(pred, axis=2)
    mask = ...
    accuracy = np.sum(outputs==test_labels)/float(np.sum(mask))

    return accuracy
```

Siamese Networks

Siamese networks are useful to identify similar questions or any similar peice of text. Platforms like Stackoverflow or Quora uses this sort of algorithm to identify duplicate questions.

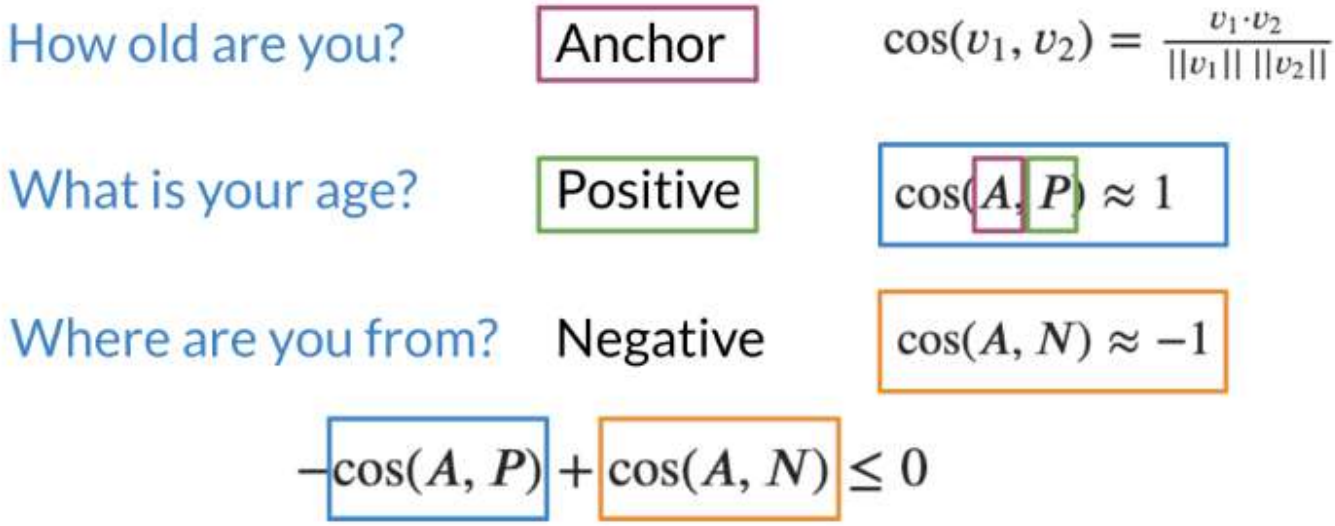
Model Architecture



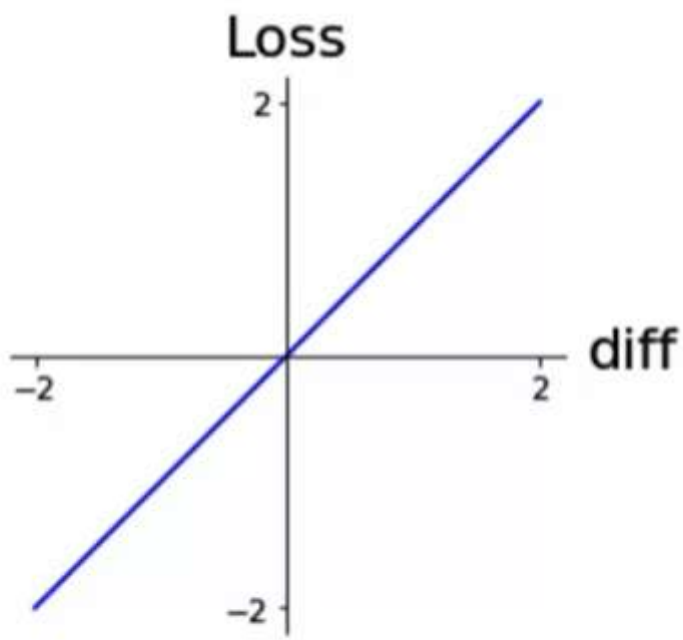
These two sub-networks are sister-networks which come together to produce a similarity score. Not all Siamese networks will be designed to contain LSTMs. One thing to remember is that sub-networks share identical parameters. This means that you only need to train one set of weights and not two.

The output of each sub-network is a vector. You can then run the output through a cosine similarity function to get the similarity score

Loss function in Siamese Networks



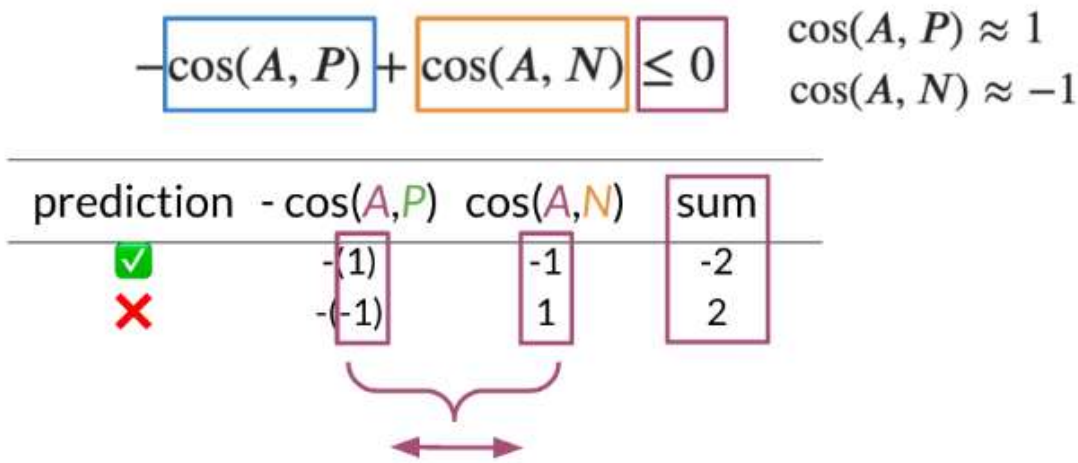
Note that when trying to compute the cost for a siamese network we use the triplet loss. The triplet loss usually consists of an Anchor and a Positive example. Note that the anchor and the positive example have a cosine similarity score that is very close to one. On the other hand, the anchor and the negative example have a cosine similarity score close to -1



$$\text{diff} = \text{s}(A, N) - \text{s}(A, P)$$

Note that if $\cos(A, P)$ =1 is 1 and $\cos(A, N)$=−1, then the equation is definitely less than 0. However, as cos(A,P) deviates from 1 and cos(A,N) deviates from -1, then you can end up getting a

cost that is > 0. Here is a visualization that would help you understand what is going on



Triplets

Q1 = How old are you ?

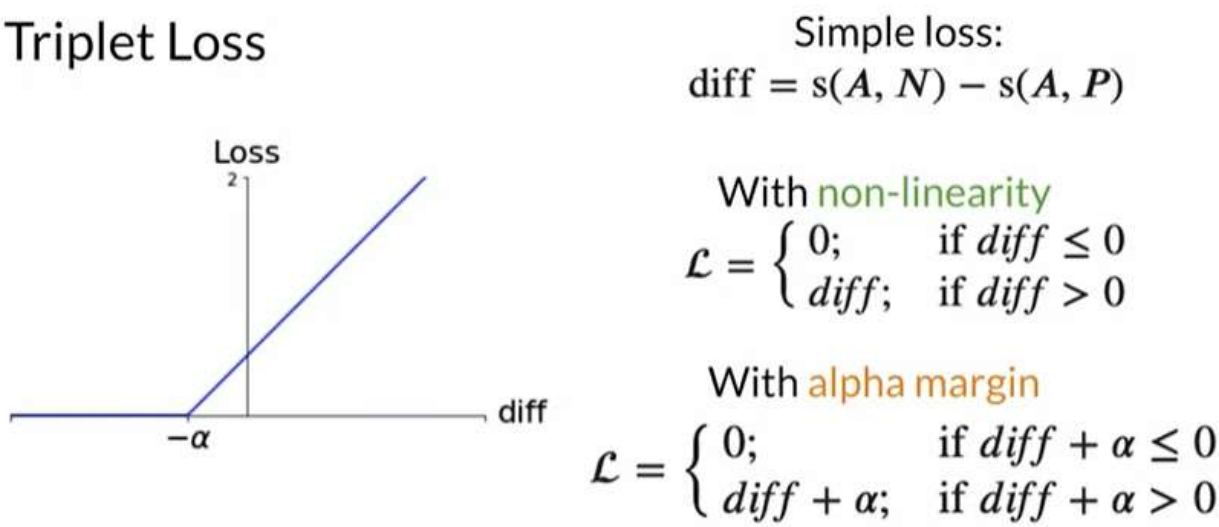
Q2 = What's your age ?

Q3 = Where have you been ?

Here, lets take Q1 to be the **Anchor**. Then Q2 will be the **Positive Question** as Q1 and Q2 carry similar meanning and Q3 would be a **Negative Question** as Q1 and Q3 do not have similarity in meaning.

The group of anchor, positive and negative questions grouped together are called **Triplet**.

The intuition is to increase the difference between the Anchor and the positive examples and the decrease the similarity between the Anchor and the Negative examples.



Note that we added an \alphaα in the equation above. This allows you to have a margin of "safety". When computing the full cost, we take the max of that the outcome of −cos(A,P)+cos(A,N)+α and 0. Note, we do not want to take a negative number as a cost

Here is a quick summary:

- α: controls how far $\cos(A,P)$ is from $\cos(A,N)$
- **Easy** negative triplet: $\cos(A,N) < \cos(A,P)$
- **Semi-hard** negative triplet: $\cos(A,N) < \cos(A,P) < \cos(A,N) + \alpha$
- **Hard** negative triplet: $\cos(A,P) < \cos(A,N)$

Triplet Selection

Triplet A, P, N
- duplicate set: A, P
- non-duplicate set: A, N

$$\mathcal{L} = \max(diff + \alpha, 0)$$

Random

$$\text{diff} = \text{s}(A, N) - \text{s}(A, P)$$

Easy to satisfy. Little to learn

$$\text{s}(A, N) \approx \text{s}(A, P)$$

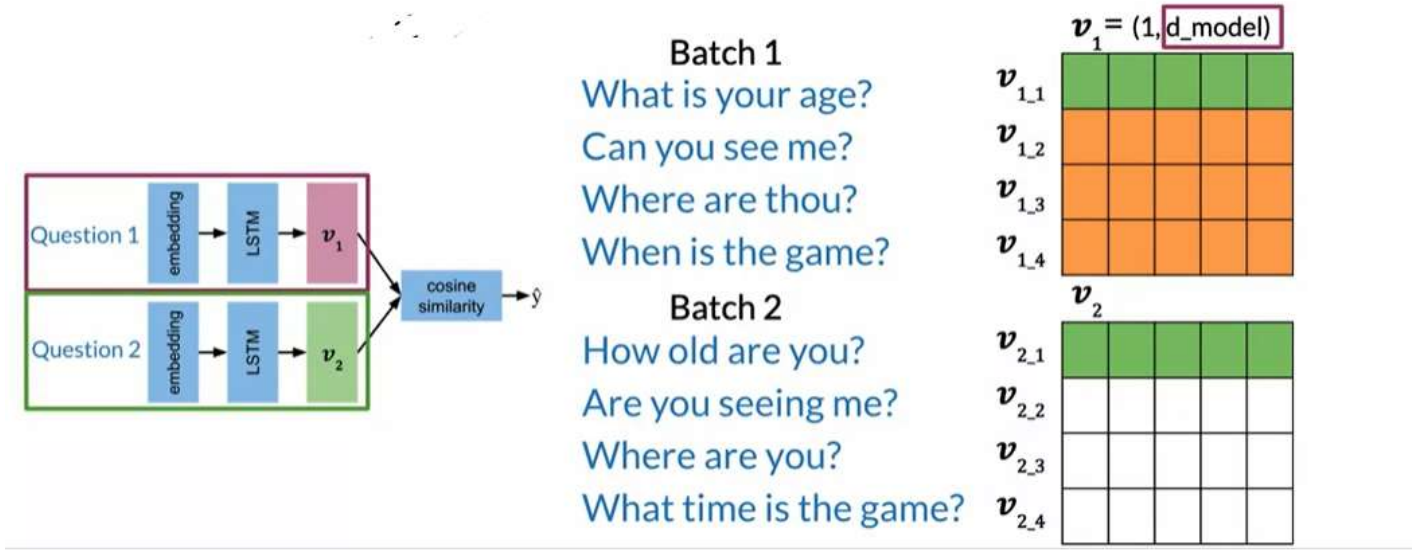
Hard

Harder to train. More to learn

Hard triplets imposes more challenges to learn the similarities between the anchor and the P and N sentences. So the more hard triplets we use to train the model the better will be the model performance eventually.

Cost Computation in Siamese Networks

d_model is a hyperparameter which is the dimension of the embedding layer of the model. d_layer determines the dimensions of the weights through each layers and thus determines the size of the output layer.
The number of rows in the metrix v1 and v2 are the batch size.Here we have 4 questions in the batch so the number of rows is also 4.V1_1 and V2_1 are duplicate questions so they are represented in green and rest of the pairs are non-duplicates so they are represented by red.



After we complete the step above we will have a matrix of dimension (batchsize*batchsize) where the diagonal elements will give us the similar questions and the off-diagonal elements will give us the questions that are not similar.If the model is performing good then the diaginal elements should have greater values than the off diagonal elements.
It's not like the similarity between the duplicate questions i.e the values of the diagonal elements has to be greater than 0 all the time. It's that the values of the diagonal elements has to be greater than the off diagonal elements. If the value of a diagonal elemnt is -0.2 and the off diagonal elements have values <-0.2 then we can say that the model is performaing good.

$$s(v_1, v_2)$$

v_2 \ v_1	_1	_2	_3	_4
_1	0.9	-0.8	0.3	-0.5
_2	-0.8	0.5	0.1	-0.2
_3	0.3	0.1	0.7	-0.8
_4	-0.5	-0.2	-0.8	1.0

$$\boxed{\begin{gathered}\mathcal{L}(A, P, N) = \max(diff + \alpha, 0) \\ \text{diff} = s(A, N) - s(A, P)\end{gathered}}$$

$$\boxed{\mathcal{J} = \sum_{i=1}^{m} \mathcal{L}(A^{(i)}, P^{(i)}, N^{(i)})}$$

To increase the performance of the model we can perform the following:

1. Mean Negative:
   Here we take the mean of the off diagonal element in each row. Example : For the first row the elements are [0.9,-0.8,0.3,-0.5] so among the off diagonal elements [-0.8,0.3,-0.5], the mean is

-0.33. So here the dagonal element 0.9 is compared with the mean value -0.33 to figure out what differentiates the two questions in further iterations.This method helps the model to converge faster by reducing noise.As we are taking mean of the negatives here so the small noise of each of the negative examples cancels out.

2. Closest Negative:
   Here we take the off-diagonal value which is sloses to the value of the diagonal element for a given row and the model try to figure out what exactly are those factors which differentiates the two questions.Example : For the first row the elements are [0.9,-0.8,0.3,-0.5] so among the off diagonal elements [-0.8,0.3,-0.5] , 0.3 is closest to the diagonal element 0.9.

For a Single Batch :

$L_{Original}(A, P, N) = Cost_{Original} = max(s(A, N) - s(A, P) + \alpha, 0) = max(diff, 0)$

$L_1(A, P, N) = Cost_1 = max(mean_negative - s(A, P) + \alpha, 0)$

$L_2(A, P, N) = Cost_2 = max(closest_negative - s(A, P) + \alpha, 0)$

$L_{full}(A, P, N) = L_1 + L_2$

For all the m bathches

$J = \sum_i^m L_{full}(A^{(i)}, (B^{(i)}, (C^{(i)})$

One-Shot Learning

Imagine you are working in a bank and you need to verify the signature of a check. You can either build a classifier with K possible signatures as an output or you can build a classifier that tells you whether two signatures are the same.

Lets assume we have built a Classification model where there were K classes. Now as we introduce a new signature so it becomes k+1 classes. Here we want the model to identify that signure everytime from only one training example without retraining the model for that one single new signature.

Here,what we do is that, we compare the similarity score between the 2 signatures to check if the two signatures are same. So, here the problem-statement moved from a classification problem to a similarity problem.

If $Similarity(Sign1, Sign2) > \tau; Sign1\ and\ Sign2\ aresimilar$

If $Similarity(Sign1, Sign2) \leq \tau; Sign1\ and\ Sign2\ are\ NOTsimilar$

Shortcomings of Sequence Models

Recurrent models typically take in a sequence in the order it is written and use that to output a sequence. Each element in the sequence is associated with its step in computation time tt. (i.e. if a word is in the third element, it will be computed at t3). These models generate a sequence of hidden

states h_t, as a function of the previous hidden state h_{t-1} and the input for position t.

The sequential nature of models you learned in the previous course (RNNs, LSTMs, GRUs) does not allow for parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. In other words, if you rely on sequences and you need to know the beginning of a text before being able to compute something